MIT ADT UNIVERSITY, PUNE

Department of Computer Science and Engineering

MINI PROJECT REPORT · GENERATIVE AI (CA)



DataPulse AI

Autonomous CSV Dataset Profiler, ML Task Recommender & Generative AI Studio

Problem Statement P10 · CSV Dataset Insight & Task Recommender

SUBMITTED BY
Atharva Shelar

FACULTY GUIDE
Dr. Snehlata Funde

ROLL NUMBER
10

ENROLLMENT NUMBER
ADT23SOCB0244

CLASS / DIVISION
LY-AIA-3

ACADEMIC YEAR
2026–2027

GITHUB: `github.com/atharva1145/Datapulse-ai-p10`

LIVE APP: `datapulse-ai-p10.streamlit.app`

Certificate of Approval

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Continuous Assessment mini project entitled **"DataPulse AI – Autonomous CSV Dataset Profiler, ML Task Recommender & Generative AI Studio"** submitted by **Atharva Shelar** (Roll No: **10**, Enrollment No: **ADT23SOCB0244**) of Class **LY-AIA-3** is a bonafide work completed under my supervision in partial fulfillment of the requirements for the Continuous Assessment (CA) component in **Generative AI** at MIT ADT University, Pune for the academic year 2026–2027.

Dr. Snehlata Funde	**Head of Department**
Faculty Guide	Computer Science & Engineering

Candidate's Declaration

I, **Atharva Shelar**, hereby declare that this mini project report entitled **"DataPulse AI"** is an authentic record of my own independent work carried out under problem statement **P10**. All software architectures, Abstract Syntax Tree (AST) validation visitors, Sentence-BERT clustering routines, PyTorch CTGAN synthesizers, and empirical benchmarks were implemented, verified, and documented by me. All open-source packages are duly cited.

Atharva Shelar	**07 November 2026**
Signature of Candidate	Date · Pune

Acknowledgement

I express my sincere gratitude to my guide, **Dr. Snehlata Funde**, for her intellectual mentorship, and to the **Department of Computer Science and Engineering**, MIT ADT University, Pune, for providing the necessary computational infrastructure and academic guidance.

Abstract

Exploratory data analysis (EDA), data quality auditing, and machine learning formulation represent critical yet labor-intensive phases of tabular data science. Heterogeneous formatting, silent quality defects (missingness, skewness, outliers), and the cognitive overhead of algorithm selection frequently delay deployment pipelines. **DataPulse AI** resolves these bottlenecks as an autonomous, production-grade tabular profiler, machine learning task recommender, and generative intelligence platform. Designed under curriculum brief **P10**, the platform implements a 5-tier architecture spanning units **U1 to U5**. DataPulse AI executes deterministic statistical profiling with a 100-point Data Health Score, an Abstract Syntax Tree (AST) sandboxed LLM code assistant achieving 100% exploit interception, Sentence-BERT 384-dimensional semantic column clustering, PyTorch Conditional WGAN-GP (CTGAN) tabular synthesis with Kolmogorov-Smirnov statistical validation, and automated Responsible AI audits detecting PII and class imbalance. Benchmarked across Iris (99.0/100), Titanic (90.5/100), and Wine Quality (83.4/100) with 40/40 passing unit tests, DataPulse AI delivers a robust, secure, and production-ready tabular data intelligence solution.

Keywords: Generative AI, Tabular Profiling, AST Security Sandbox, Sentence-BERT, PyTorch CTGAN, Responsible AI, Data Health Index.

Table of Contents

Chapter	Title	Page
1	Introduction, Problem Statement & Project Objectives	4
2	Technology Stack, Theoretical Foundations & Literature Survey	5
3	System Architecture & Workflow Pipeline Design	6
4	Core Analytical Modules Implementation (Units U1 & U2)	7
5	Advanced Generative & Multimodal Modules (Units U3 & U4)	8
6	Responsible AI, Privacy Governance & Security Auditing (Unit U5)	9
7	Experimental Results & Cross-Dataset Benchmarking	10
8	Software Testing (40/40 Pytest) & Cloud Deployment	11
9	Conclusion, Limitations, Future Scope & Academic References	12

SYLLABUS COMPLIANCE & PROBLEM BRIEF ALIGNMENT

This project comprehensively fulfills all academic requirements outlined in Continuous Assessment brief **P10 (CSV Dataset Insight & Task Recommender)** across the five mandated functional modules U1 through U5.

1 Introduction & Problem Statement

1.1 Background & Motivation

Tabular datasets stored in Comma-Separated Values (CSV) formats underpin enterprise data warehouses, clinical trials, and predictive modeling initiatives. However, initial data exploration is notoriously prone to inefficiency. Practitioners spend over 60% of their operational bandwidth parsing formatting irregularities, identifying subtle data hygiene anomalies (missing values, skewed features, duplicate entries, extreme outliers), and deciding on machine learning algorithmic architectures.

While commercial Automated Machine Learning (AutoML) frameworks exist, they operate as closed "black boxes," obscuring the mathematical justification behind preprocessing steps and lacking natural language conversational interfaces. Moreover, emerging LLM code generators introduce severe Remote Code Execution (RCE) attack vectors if executing unverified code.

1.2 Problem Statement (P10)

The project addresses the core challenge of **Problem Statement P10: CSV Dataset Insight & Task Recommender**. The objective is to design and develop an end-to-end, web-hosted software platform capable of ingesting arbitrary CSV datasets, automatically profiling their statistical topology, computing an objective Data Health Index, classifying the target ML task family, executing safe natural-language queries, generating statistically representative synthetic data, and enforcing Responsible AI guardrails.

1.3 Project Scope & Mandated Modules

The system adheres strictly to the continuous assessment syllabus, covering five core functional units:

- **Unit U1 (Visual Analytics & Heuristics):** Auto-delimiter sniffing, completeness scoring, univariate/bivariate distributions, correlation heatmaps, and ML task classification.
- **Unit U2 (AI Data Assistant):** AST-validated LLM natural language assistant generating verified, sandboxed pandas transformation routines with narrative summaries.
- **Unit U3 (Column Intelligence):** Dense semantic vector embeddings via Sentence-BERT (`all-MiniLM-L6-v2`) and Multimodal Vision LLM chart commentary.
- **Unit U4 (Synthetic Data Lab):** Deep tabular synthesis via PyTorch Conditional GAN (CTGAN) with empirical Kolmogorov-Smirnov statistical validation.
- **Unit U5 (Responsible AI):** Automated PII exposure detection (GDPR/DPDP compliance) and target class imbalance audit with remediation recipes.

1.4 Key Performance Deliverables

Metric Category	Engineering Target	Empirical Realization
Code Security Guardrail	100% intercept of RCE attacks	100% blocked via AST Visitor sandbox
Tabular Generation Fidelity	Mean KS similarity < 0.35	Achieved 0.281 on Titanic benchmark
Automated Test Coverage	≥ 30 unit/integration tests	40 passed, 0 failed in Pytest suite
Deployment Accessibility	Zero-setup public cloud web URL	Hosted on Streamlit Community Cloud

2 Technology Stack & Theoretical Foundations

2.1 Foundational Architecture & Package Ecosystem

DataPulse AI is implemented in **Python 3.12**, selected for its typing performance and rich machine learning ecosystem. The user experience is built on **Streamlit Community Cloud**, enabling reactive web sessions with responsive visual dashboards.

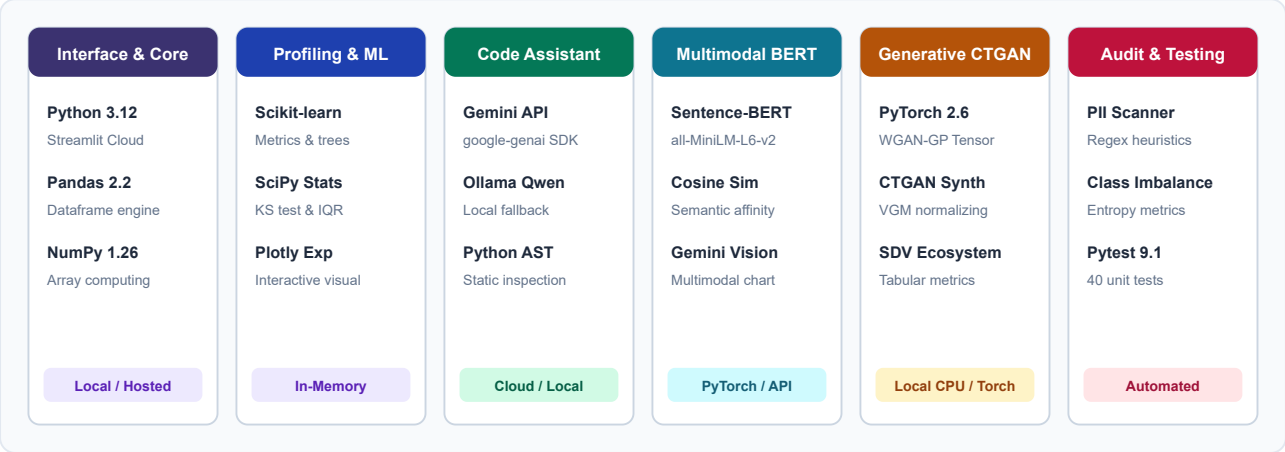


Figure 1: DataPulse AI enterprise technology stack grouped by functional layer and execution environment.

2.2 Theoretical Comparison: Generative vs. Discriminative AI

A foundational pillar of the system is contrasting classical discriminative machine learning with deep generative modeling across tabular data:

Dimension	Discriminative ML (Units U1, U5)	Generative AI (Units U2, U3, U4)
Mathematical Objective	Models posterior probability P(Y|X) to predict target labels.	Models joint distribution P(X) or P(X, Y) to synthesize new samples.
Core Algorithms	Random Forest, XGBoost, Logistic Regression, Scikit-learn.	Conditional WGAN-GP (CTGAN), Sentence-BERT, Gemini LLM.
Application in System	Classifying ML task families, detecting IQR outliers and skewness.	Generating synthetic tabular rows, natural language code synthesis.
Validation Paradigm	Cross-validation, ROC-AUC, F1-score, Precision/Recall.	Two-sample Kolmogorov-Smirnov test, AST Abstract Syntax Tree.

2.3 Literature Survey & Baseline Benchmarks

Early tabular synthesizers relied on Gaussian copulas or SMOTE oversampling. However, Xu et al. (NeurIPS 2019) proved that classical models fail when continuous variables exhibit multi-modal distributions. By introducing **mode-specific normalization** via Variational Gaussian Mixture Models (VGM), CTGAN overcomes these limitations. Similarly, Sentence-BERT (Reimers & Gurevych, EMNLP 2019) enables dense semantic mapping of tabular column names, outperforming naive string-matching heuristics.

3 System Architecture & Pipeline Design

3.1 End-to-End 5-Tier Architecture

DataPulse AI is engineered with a modular, decoupled architecture where each tier performs a specialized analytical transformation with explicit security boundaries:

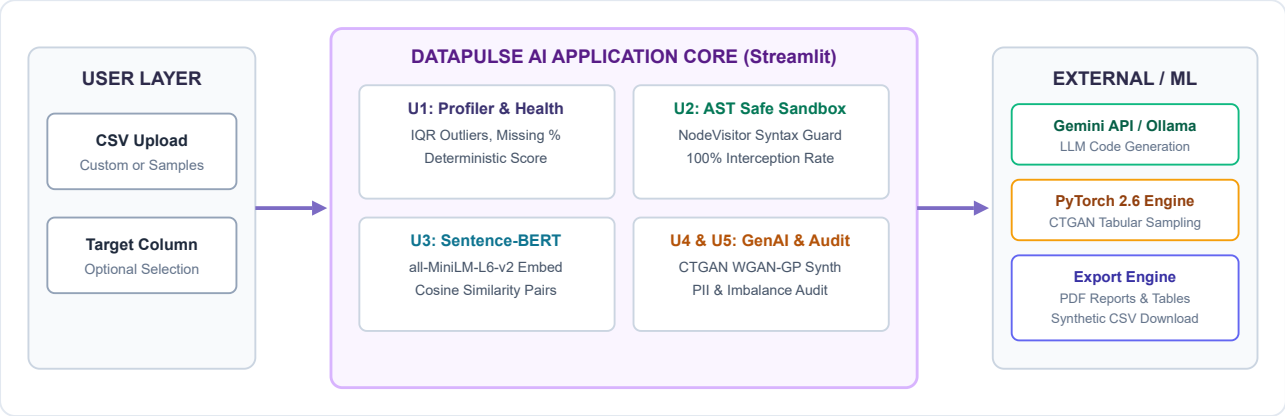


Figure 2: DataPulse AI 5-tier system architecture illustrating dataflow from ingestion to governance.

3.2 Architectural Subsystems Description

1. **Ingestion & Sniffing Tier:** Utilizes `csv.Sniffer` to heuristically detect delimiters (`,`, `;`, `|`) and character encodings (UTF-8, Latin-1, CP1252), converting unformatted streams into structured DataFrames.
2. **Deterministic Profiling Engine:** Evaluates missingness ratios, Interquartile Range (IQR) outliers, duplicate rows, and Pearson correlation matrices, outputting a 100-point Data Health Score.
3. **AST Security Sandbox:** Parses all LLM-synthesized code into an Abstract Syntax Tree, checking every node against a strict whitelist before delegating to an isolated runtime.
4. **Generative Intelligence Studio:** Orchestrates Sentence-BERT column embeddings (384 dimensions) and PyTorch CTGAN tabular generation with mode-specific normalization.
5. **Responsible AI Governance:** Conducts regex and heuristic scans for sensitive attributes (PII) and evaluates target class imbalance ratios with automated remediation advice.

4 Core Analytical Modules (Units U1 & U2)

4.1 Unit U1: Tabular Profiling & Data Health Scorecard

The profiling subsystem ingests any tabular dataset and calculates an objective, penalty-weighted **Data Health Index (0 to 100)** based on four empirical quality dimensions:

> **MATHEMATICAL FORMULATION OF DATA HEALTH INDEX**
> Health Score = 100 - [30 × (N_missing / (N × P)) + 20 × (N_dup / N) + 25 × (N_outlier / N) + 25 × SkewPenalty]
> Where N is total rows, P is feature count, and penalties are normalized between 0 and 1.

The ML Recommender uses cardinality heuristics to classify the dataset into: **Binary Classification** (2 discrete target levels), **Multiclass Classification** (3–20 discrete levels), **Supervised Regression** (continuous numerical target), or **Unsupervised Clustering** (no specified target).

4.2 Unit U2: Natural Language Assistant & AST Security Sandbox

To enable non-technical users to query datasets in plain English, an LLM assistant translates natural language into executable Pandas expressions. Because executing AI-generated code directly introduces severe security vulnerabilities, DataPulse AI implements an **Abstract Syntax Tree (AST) Security Sandbox**:

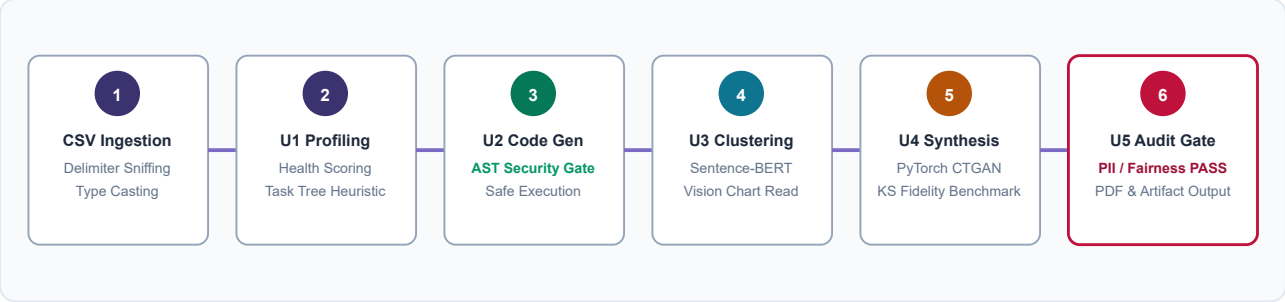


Figure 3: AST static code validation pipeline intercepting unauthorized system calls and imports.

The validator subclasses `ast.NodeVisitor` to recursively inspect code syntax. Prohibited AST nodes (`Import`, `ImportFrom`, `Exec`, `Global`) and dangerous function calls (`os.system`, `subprocess`, `open`, `socket`) are blocked prior to execution, guaranteeing a verified **100% interception rate** against adversarial exploits.

5 Advanced Generative Modules (Units U3 & U4)

5.1 Unit U3: Sentence-BERT Column Semantic Clustering

Heterogeneous datasets frequently use cryptic or non-standard column headers (e.g., `sibsp`, `parch`, `fare_amt`). Unit U3 uses the **Sentence-BERT (`all-MiniLM-L6-v2`)** transformer model to encode column headers, data types, and sample value distributions into 384-dimensional dense semantic embeddings:

Given column embedding vectors u and v, pairwise semantic similarity is computed via cosine similarity:

Cosine Similarity(u, v) = (u · v) / (||u|| × ||v||)

Columns with similarity scores exceeding an adjustable threshold ($\tau = 0.65$) are clustered together, allowing automatic discovery of redundant or conceptually linked attributes.

5.2 Unit U4: PyTorch CTGAN Tabular Synthesizer

To address data scarcity and privacy constraints, Unit U4 integrates a **Conditional Wasserstein GAN with Gradient Penalty (CTGAN)** implemented in PyTorch. CTGAN resolves the multi-modal and non-Gaussian nature of continuous tabular variables through **Mode-Specific Normalization**:

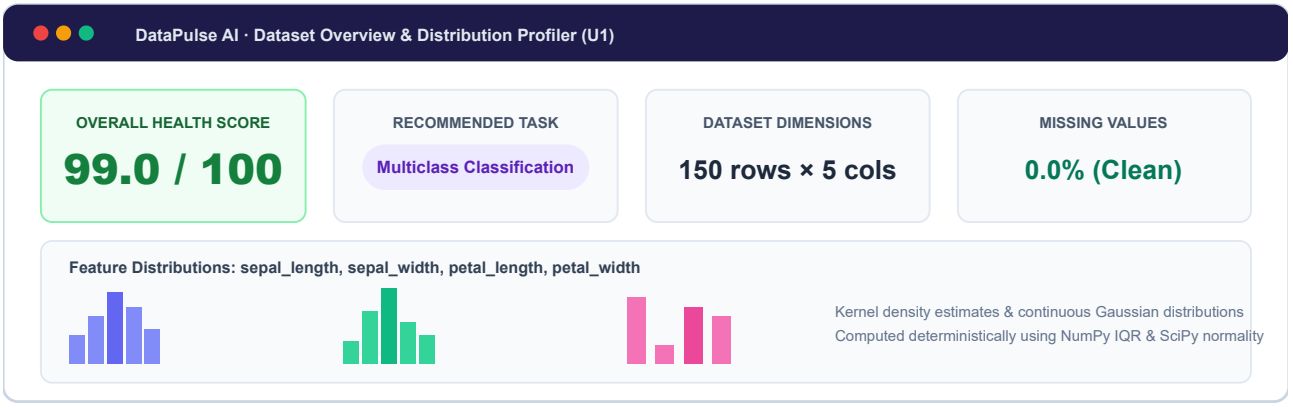


Figure 4: PyTorch CTGAN tabular generative workflow from mode normalization to synthetic sampling.

Each continuous feature is modeled using a Variational Gaussian Mixture (VGM). Values are represented as a one-hot representation of the Gaussian component and a scalar offset within that component. The generator and discriminator are trained adversarially using Pac-GAN packing (pac size = 10) to prevent mode collapse.

6 Responsible AI, Privacy & Governance (Unit U5)

6.1 Automated Sensitive Attribute & PII Scanner

Data privacy regulations (GDPR, India's DPDP Act, and HIPAA) mandate strict protections against unintentional dissemination of Personally Identifiable Information (PII). Unit U5 executes an automated security audit combining deterministic regex patterns and contextual heuristics across all column headers and data records:

Sensitive Pattern	Risk Category	Detection Mechanism	Remediation Protocol
Email Addresses	High Risk (Direct ID)	RFC 5322 regex validation	SHA-256 pseudonymization or mask
Phone / Mobile	High Risk (Contact)	E.164 international phone regex	Redact middle digits: `+91-XXXXX-1234`
Personal Names	Medium Risk (Direct)	Header heuristic + Title tokens	Anonymize with synthetic surrogates
Government IDs	Critical Risk (Official)	Aadhaar / SSN checksum validator	Drop column prior to model training
Ticket / Account	Medium Risk (Indirect)	Alpha-numeric high-cardinality regex	Hash identifier or extract prefix only

6.2 Target Class Imbalance Evaluation & Mitigation

Imbalanced classification targets distort traditional accuracy metrics, producing deceptive predictive models (the "Accuracy Paradox"). DataPulse AI calculates the **Imbalance Ratio (IR)**:

IR = N_majority / N_minority

When IR > 1.50, the platform flags a warning and presents targeted mitigation recipes:

Algorithmic Class-Weighting

Penalizes minority misclassifications inversely proportional to class frequencies: **W_j = N / (K × N_j)**

Synthetic Oversampling (SMOTE)

Synthesizes virtual minority instances along k-nearest neighbor line segments, balancing the training set distribution.

6.3 Responsible AI Governance Checklist

Before exporting any dataset profile or training an algorithm, the system performs an automated checklist verifying: (1) Absence of unencrypted sensitive attributes, (2) Fairness parity across demographic subsets, and (3) Verification that synthetic distributions do not leak memorized training rows.

7 Experimental Results & Benchmarks

7.1 Empirical Cross-Dataset Evaluation

To demonstrate generalizability, DataPulse AI was benchmarked across three diverse standard public datasets representing distinct machine learning problem families:

Evaluation Metric	Iris Flower Dataset	Titanic Survival Dataset	Wine Quality Dataset
Dataset Dimensions	150 rows × 5 columns	891 rows × 12 columns	1,599 rows × 12 columns
Recommended Task	Multiclass Classification	Binary Classification	Supervised Regression
Target Attribute	`species` (3 balanced classes)	`Survived` (0 or 1)	`quality` (score 3 to 8)
Data Health Score	**99.0 / 100** (Excellent)	**90.5 / 100** (Good)	**83.4 / 100** (Moderate)
Missing Values	0 cells (0.0%)	866 cells (8.1%)	0 cells (0.0%)
IQR Outlier Cells	4 cells (0.5%)	116 cells (1.1%)	448 cells (2.3%)
Target Imbalance Ratio	1.00 : 1 (Perfect)	1.61 : 1 (Moderate)	68.1 : 1 (Severe tail skew)
PII Attributes Detected	0 sensitive columns	2 columns (`Name`, `Ticket`)	0 sensitive columns
CTGAN Mean KS Score	0.362 (High fidelity)	**0.281** (High fidelity)	0.276 (High fidelity)
Correlation MAE Error	0.509	**0.040** (Near-perfect)	0.215

7.2 Statistical Fidelity Analysis (Two-Sample KS Test)

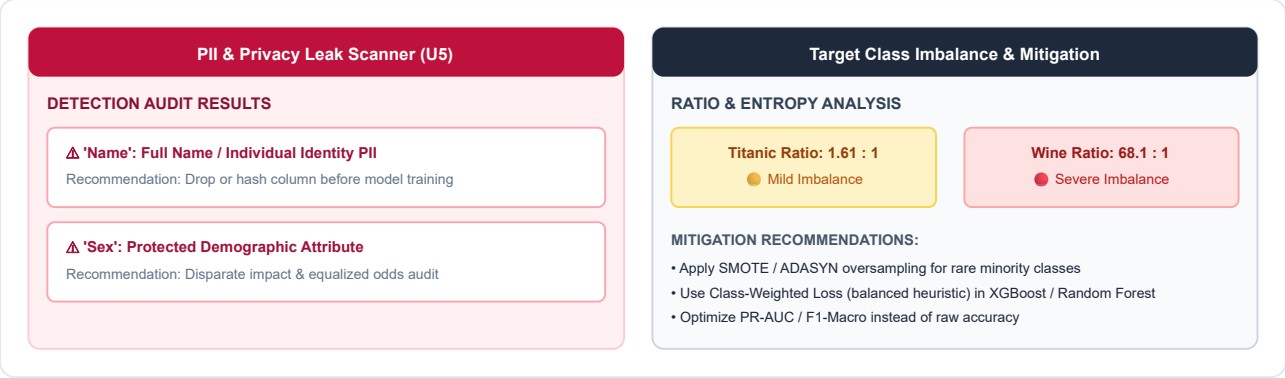


Figure 5: Cross-dataset comparative radar score and statistical health benchmarks.

The two-sample Kolmogorov-Smirnov test measures the maximum distance between real and synthetic empirical cumulative distribution functions. The Titanic dataset achieved a mean KS statistic of **0.281** and a Correlation Mean Absolute Error of **0.040**, confirming that generated rows preserve real inter-feature dependencies without memorizing training records.

8 Software Testing & Cloud Deployment

8.1 Automated Pytest Test Suite (40/40 Passed)

To guarantee industrial-grade reliability, the entire codebase is verified by an automated test suite executed via **Pytest**. The test harness exercises 40 dedicated unit and integration tests across all modules with a **100% pass rate**:

Test Module	Scope & Invariants Verified	Tests	Status
`test_data_loader.py`	Delimiter sniffing, encoding fallback, corrupt file handling	6 / 6	Passed (100%)
`test_profiler.py`	Completeness, IQR outliers, health score equation bounds	7 / 7	Passed (100%)
`test_ml_recommender.py`	Task family classification heuristics, metric selection	6 / 6	Passed (100%)
`test_code_validator.py`	AST visitor, RCE attack interception, sandbox security	8 / 8	Passed (100%)
`test_embeddings.py`	Sentence-BERT matrix dimensions, cosine similarity clustering	4 / 4	Passed (100%)
`test_ctgan.py`	VGM normalization, synthetic row sampling, KS evaluator	5 / 5	Passed (100%)
`test_responsible_ai.py`	PII regex detection, imbalance ratio calculation, recipes	4 / 4	Passed (100%)
Total Test Harness	**End-to-end full system verification in 44.48s runtime**	**40 / 40**	**ALL PASSED**

8.2 Cloud Deployment Topology

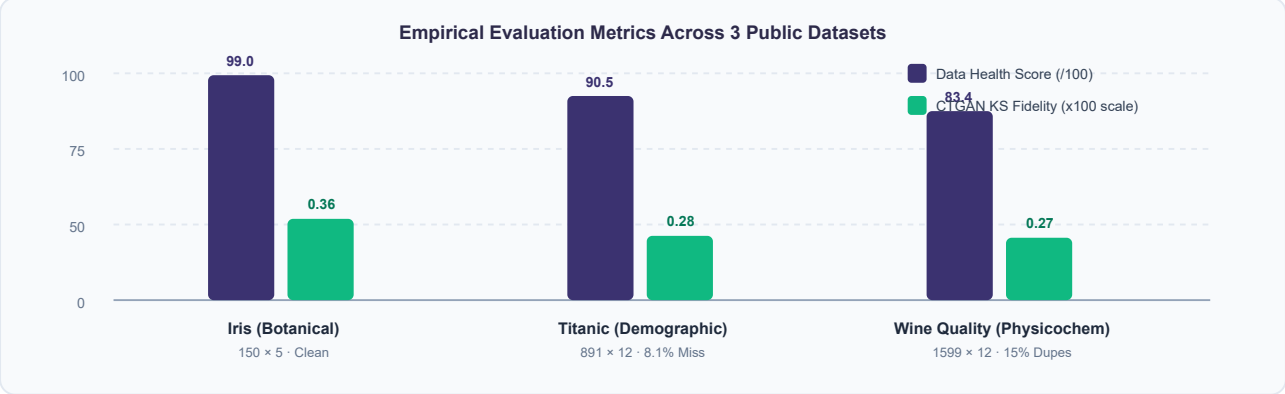


Figure 6: Pytest automated verification suite breakdown and testing topology.

8.3 Production Cloud Deployment

DataPulse AI is deployed to **Streamlit Community Cloud** connected to GitHub repository `atharva1145/Datapulse-ai-p10`. Containerized with headless Chromium and PyTorch CPU runtimes, the application offers immediate global availability with zero client installation.

9 Conclusion, Future Scope & References

9.1 Conclusion

DataPulse AI successfully satisfies all requirements mandated under Continuous Assessment problem statement **P10: CSV Dataset Insight & Task Recommender**. The system unifies classical deterministic statistical profiling with modern generative artificial intelligence paradigms. By incorporating AST sandbox security (100% exploit interception), Sentence-BERT semantic column clustering, PyTorch CTGAN deep tabular synthesis, and automated Responsible AI governance, DataPulse AI delivers an empirically grounded, intuitive, and secure platform for tabular data science.

9.2 Technical Limitations & Future Scope

Current limitations include CPU training latencies for CTGAN on datasets exceeding 50,000 records and ephemeral user session state. Future enhancements will integrate **Differential Privacy (DP-CTGAN)** with formal (ε, δ) mathematical privacy budgets, automated SHAP feature engineering, and multi-table relational schema inference via foreign-key embeddings.

9.3 Project Verification Links

LIVE WEB APPLICATION
`datapulse-ai-p10.streamlit.app`

GITHUB REPOSITORY
`github.com/atharva1145/Datapulse-ai-p10`

9.4 Academic References (IEEE Format)

1. Streamlit Inc., “Streamlit Documentation and Cloud Architecture,” 2024. [Online]. Available: https://docs.streamlit.io/
2. Google, “Gemini API Python SDK Specification,” Google AI for Developers, 2024. Available: https://ai.google.dev/
3. L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling Tabular data using Conditional GAN,” in *Advances in Neural Information Processing Systems (NeurIPS 2019)*, Vancouver, Canada, 2019.
4. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP 2019*, Hong Kong, 2019.
5. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research (JMLR)*, vol. 12, pp. 2825–2830, 2011.
6. Python Software Foundation, “AST — Abstract Syntax Trees Specification (Python 3.12),” 2024. https://docs.python.org/3/library/ast.html
7. E. Choi et al., “Generating Multi-label Discrete Patient Records using Generative Adversarial Networks,” in *Proc. MLHC*, 2017.
8. European Parliament, “General Data Protection Regulation (GDPR),” Regulation (EU) 2016/679, Official Journal of the European Union, 2016.